



🔍 Job Search



ResuMind

Team NY4



Team Introduction



Anika Bushra
Front End Developer



Eric Lantz
Front End
Developer



Sarah Wierzbicki
Machine Learning
Trainer/Developer



Joseph Indriolo
Backend/DevOps
Engineer



Agenda

Problem Statement

Description

Personas

Technologies

Algorithms

Team Working Agreement



Problem Statement

- Whether you are actively or passively searching for a job, looking for new opportunities can be overwhelming due to:
 - High volume of listings on job sites such as Indeed, LinkedIn, Monster
 - Lack of matching/relevant skills
 - Fake job postings
- There is a clear need for a smarter, more personalized approach to job discovery—one that truly understands the applicant, while making the job selection process simple and clear.



Project Description

- ResuMind is an AI-powered job assistant that personalizes the job search experience for job seekers who struggle to find roles that match their skills and career goals. Our application uses natural language processing to recommend verified job opportunities and helps users understand how closely they fit the role. Unlike traditional job boards that require manual filtering, ResuMind offers a tailored experience by highlighting key details, comparison of strong skills versus areas of improvement, and ways to enhance your resume.
- Benefits include:
 - Reduced job search time
 - Skill match insights
 - Resume feedback
 - Interview prep

Personas



Maria Smith, Returning to Workforce Parent

- **Age:** 40
- **Background:** Former advertising professional that took an 8-year gap to raise children
- **Job search style:** Moderately active
- **Skills:** Microsoft suite, content creation, Adobe creative suite, client communication
- **Challenges:** Unsure of how to present career gap, finding jobs that match previous skills acquired, how to market herself as a competitive candidate
- **Needs:** Resume refresh guidance, remote/hybrid work environment, personalized job recommendations based on experience



John Lang, Laid-Off Senior Director

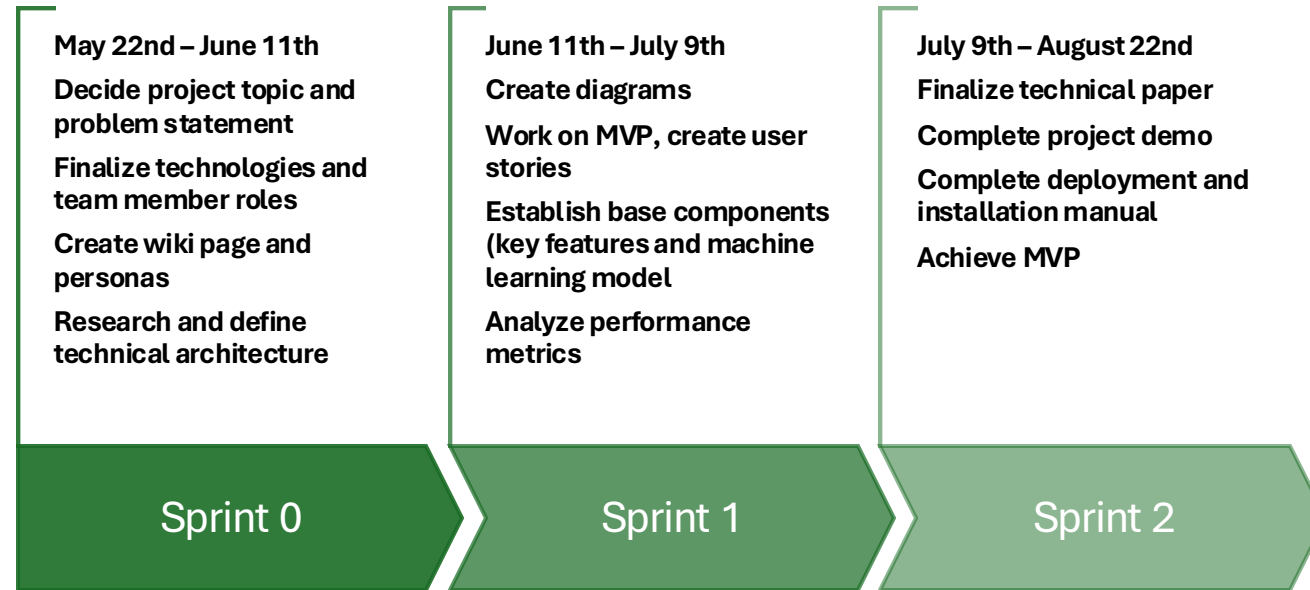
- **Age:** 51
- **Background:** Senior Software Engineer Director with 20+ years of relevant experience, recently got laid off due to company budget cuts.
- **Job search style:** Actively searching
- **Skills:** Agile/scrum leadership, cloud platforms, people leader, system design
- **Challenges:** Difficulty navigating a saturated environment, overqualified for many roles
- **Needs:** Competitive salary, curated job listings, specialized skill match



Lisa Burns, Entry-Mid Level Analyst

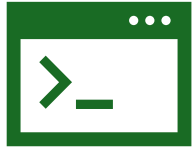
- **Age:** 28
- **Background:** 4 years as a data analyst in the healthcare industry, looking to transition to the finance industry
- **Job search style:** Passively seeking
- **Skills:** SQL, Python, Data cleaning/validation, advanced proficiency in Excel
- **Challenges:** Lack of time to seek new opportunities, irrelevant job alerts, limited knowledge of financial industry
- **Needs:** High quality job suggestions, analysis of skill transferability, access to learning resources

Project Schedule



Weekly meetings will be held at Wednesdays at 8:30 PM to review our progress together and establish weekly individual tasks

Technologies



Languages, libraries and frameworks:

- Front-end:
 - React, JavaScript, Node.js, Express.js
- Back-end:
 - Python, PyTorch, Flask
- Libraries:
 - SpaCy, Scikit-Learn, HuggingFace



Database Management

- MongoDB, AWS



Tools

- Google Colab
- WhatsApp
- Microsoft Teams
- GitHub
- Jira
- VSCode
- Amazon EC2



Algorithms & Design

- **Natural Language Processing**
 - Using the open-source library SpaCy, performing named entity recognition (NER) for keyword extraction
 - Resumes will be parsed using this process to extract valuable text.
- **Text - Classification Model**
 - Our fine-tuned model, which uses RoBERTa as the base model, was trained on 2500 resumes with the goal of text classification.
 - This model will then make a prediction on the extracted text and create a user category based on their skills and experience

Team Working Agreement

NY4 Team Working Agreement

June 7, 2025

Team Members

- Joseph Indriolo
- Anika Bushra
- Eric Lantz
- Sarah Wierzbicki

Communication

The team has agreed to make WhatsApp our main channel of text communication. The platform is easily accessible by both mobile and desktop. This will serve as the primary platform to ask questions, raise concerns, or provide any updates. In addition to WhatsApp, the team will use Microsoft Teams as our main platform for live video calls. Live video calls will be scheduled Monday and Wednesday each week at 8:30 PM EST. During the live meetings, we will share any updates with the team, and hold important agile ceremonies as needed. We will schedule more time throughout the week as needed in order to meet our deliverable deadlines. All members of the team are encouraged to contribute to all conversations, as our intention is to deliver the best possible product and diverse experiences and viewpoints are the best way to do this.

Individual Responsibilities

During Sprint planning ceremonies, each team member will be assigned a ticket based on their preference and skill. During planning, each ticket will be pointed in terms of complexity and assigned to an individual to complete. When a ticket is assigned, the assignee is responsible for owning and delivering that work end-to-end. Each team member is expected to deliver their work as committed to in sprint planning. Sprints will be carefully planned according to the team's velocity/capacity. In the event a ticket cannot be completed in the allocated sprint, the ticket will be rolled over to the next sprint.

Definition of Done (DOD)

A ticket is only considered done when the following criteria is met:

- All the acceptance criteria defined in the ticket have been completed
- The feature was tested, fully working and free of errors and bugs.
- The work has been peer reviewed by the rest of the team.
- The feature has been demoed and shared with the rest of the team.
- If it is a code change, the PR has been approved, and merged into the main branch.
- Any acceptable documentation has been updated.

Team Values

Each team member is responsible for creating a positive and productive working environment. Criticism is encouraged if it is constructive and genuinely intended to improve the project, or the individual's skills. Each team member is encouraged to call out any concerns or blockers early and as needed. Communication is very important and each developer is expected to call out early if they do not believe they can deliver the work they have committed to. Personal attacks or the creation of a toxic working environment are not permitted and will be reported as necessary.

Team Working Agreement



Sprint 2 Agenda

- Sprint 1 Recap
- MVP
- Improvements
- **Diagrams** (Architecture Diagrams, Context Diagram, ER Diagrams, Sequence Diagram, State Diagrams, Class Diagrams)
- Product Backlog
- Sprint 2 User Stories & Acceptance Criteria
- **Metrics** (Team velocity, burndown chart, completed/committed ratio)
- Project Demo
- Website Screenshots
- Test cases
- Sprint 2 Backlog
- Retrospective



Sprint 1 Recap

- **Frontend Development**

- Set up the React frontend environment.
- Created placeholder pages/components such as the home screen, resume uploader, and navigation bar.

- **Backend Development**

- Created text preprocessing module
- Trained Roberta model for text classification to predict job category
- Used model on job listing database for inference to find category to eventually match with user resume
- Set up the backend framework, Flask

MVP

Our vision: Helping job seekers efficiently find relevant opportunities by scanning their resumes for keywords and experience, then offering a simple swipe-based system to explore those matches.

Core Functionality:

- Resume Upload: Users upload their resume directly onto the platform.
- Keyword-Based Resume Parsing: Natural language processing extracts information from resumes and our machine learning model determines a relevant job category based on the resume.
- Job Search & Swipe Interface: Based on content from the parsed resume, a curated list of relevant jobs is generated. Users can swipe right to like or left to dismiss – like a dating app, but for job hunting instead.
- Matched Jobs List: Liked jobs are saved to a Matched Jobs page where users can review a match percentage and the application.

Improvements



Ensure clear audio and video resolution across all recordings



Add dedicated slide for Sprint backlog and include story point estimations directly in the backlog



Rewrite all user stories from the user's perspective with proper acceptance criteria and clear benefit/value

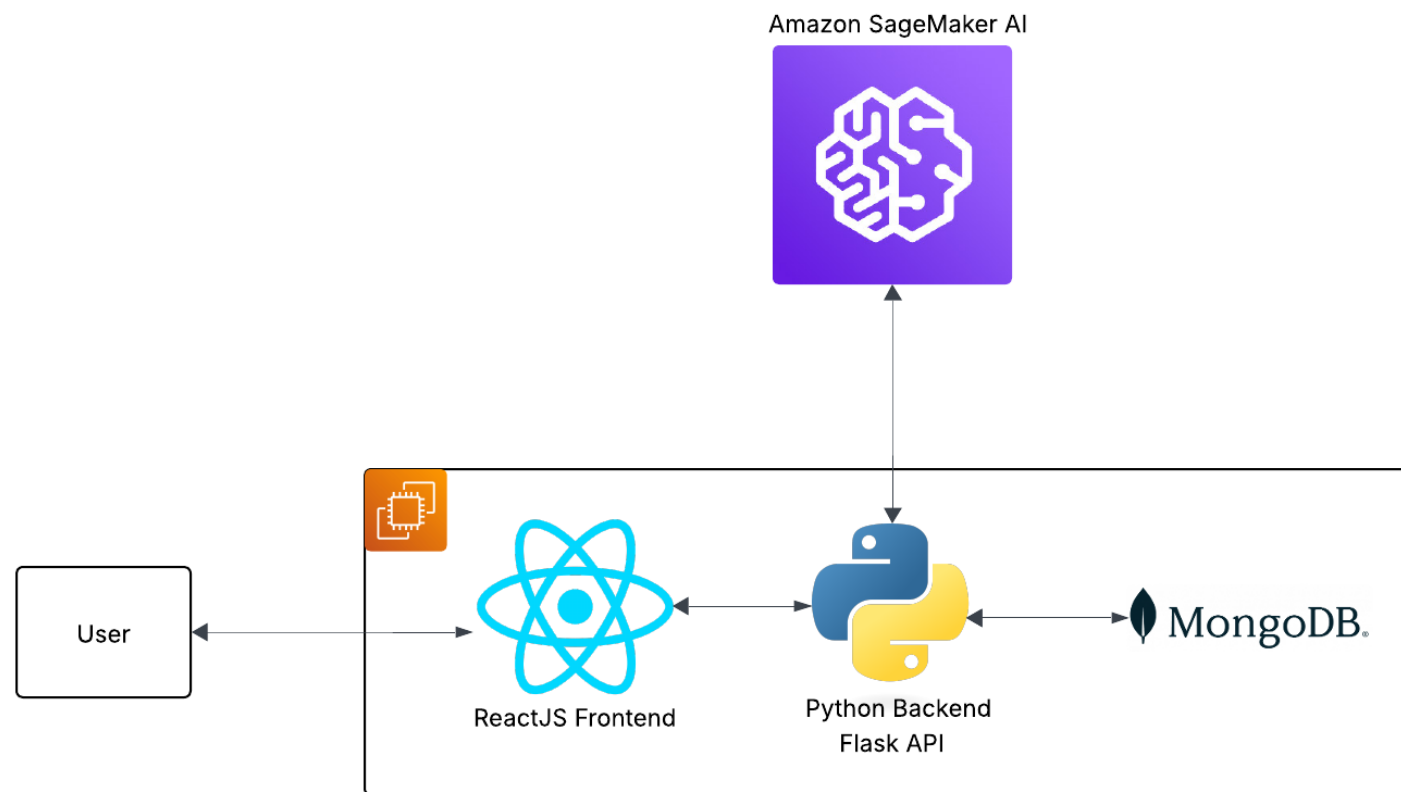


Present user stories (not subtasks) to show functionality completed and clarify the difference between the story, task, and subtask

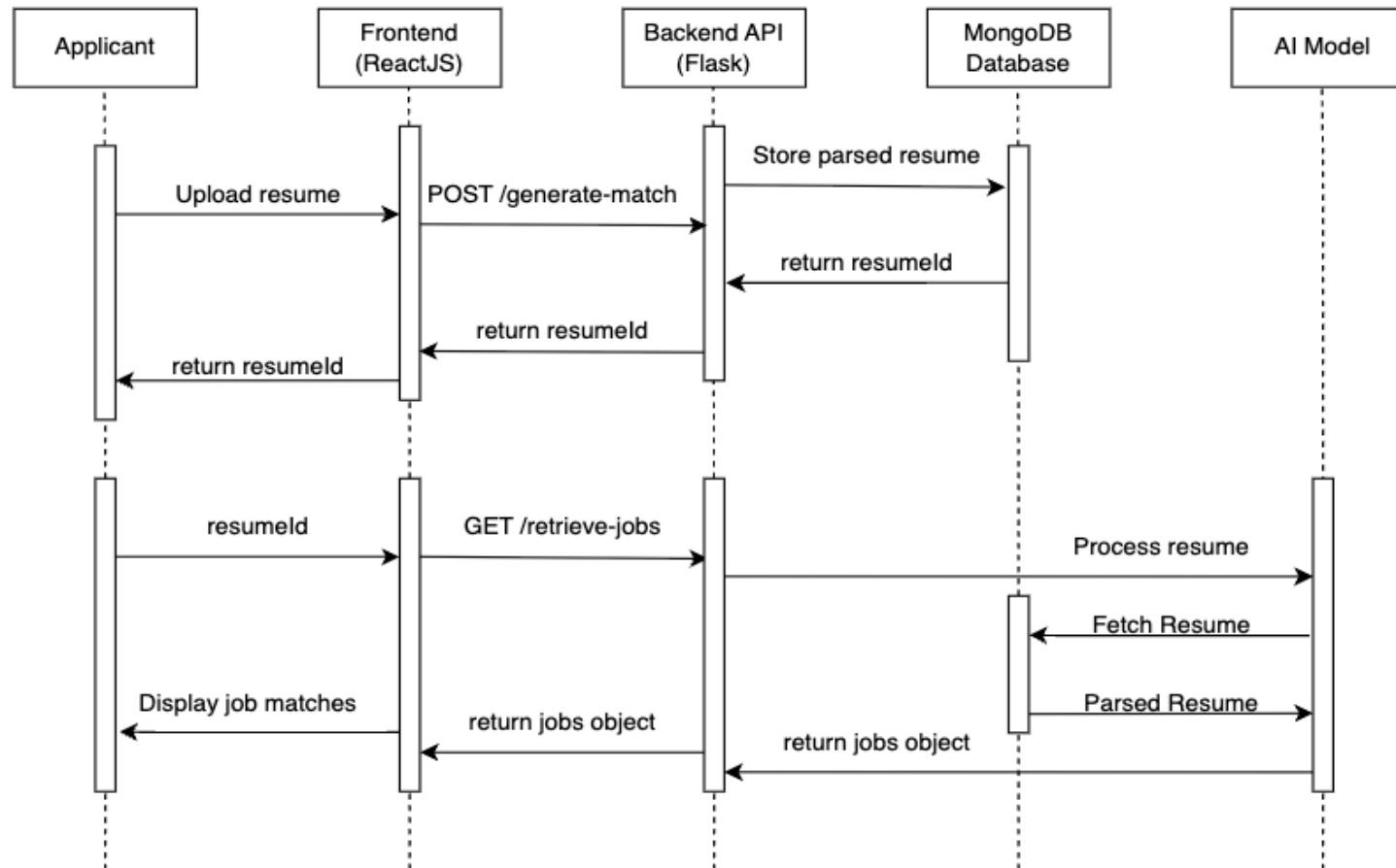


Present together as a team for consistency and continuity

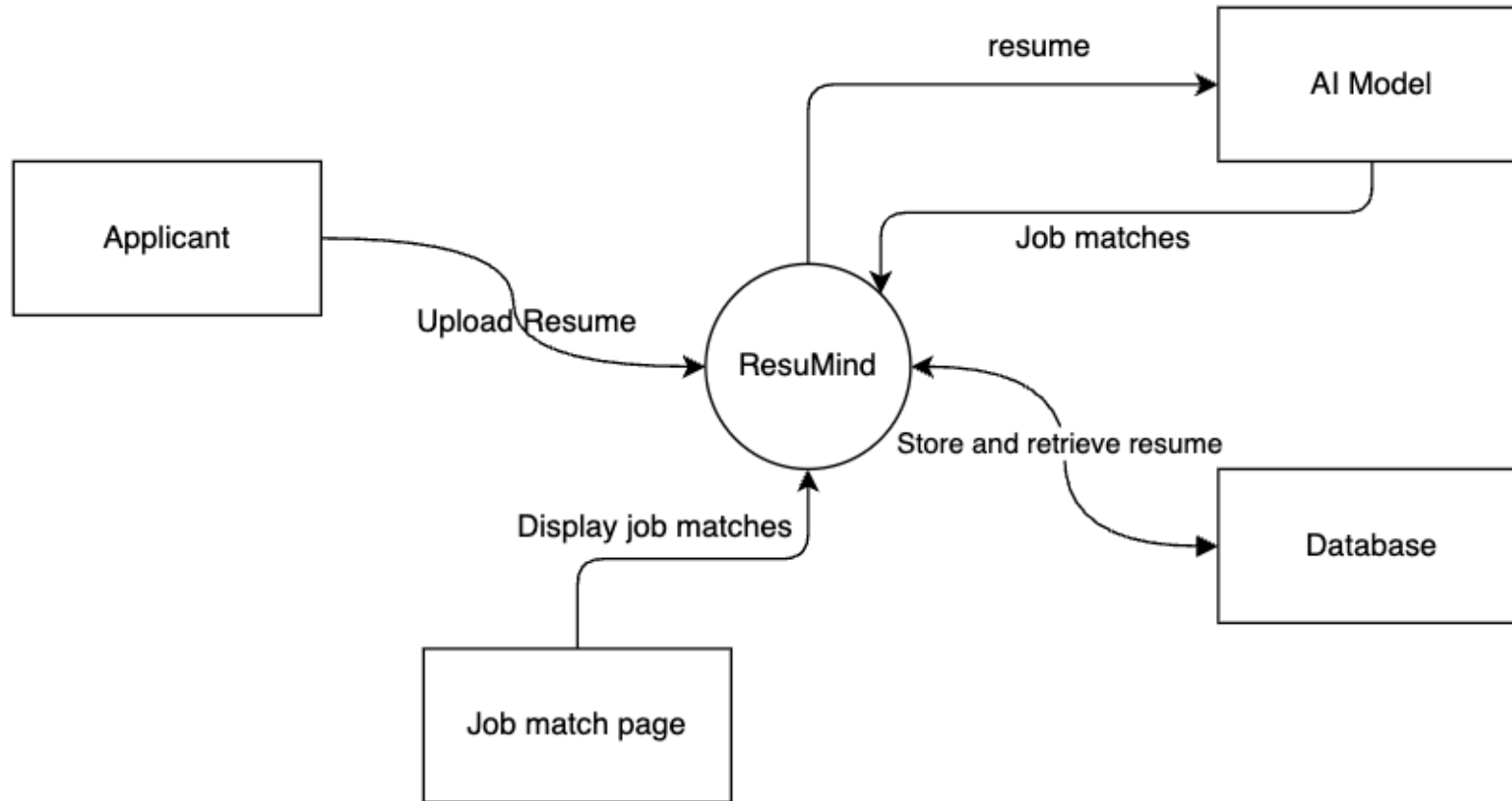
Architecture Diagram



Sequence Diagram



Context Diagram



Product Backlog

Issue key	Summary	Description	Assignee	Status	Comments	(Story point estimate)	Sprint
NY4-57	Create GET Model Inference	User Story: * As a job seeker, I want the platform to generate job recommendations based on my resume's content, so that I can see how well I match open roles. Acceptance Criteria: * A GET endpoint /model-inference accepts a resumeID as part of a body. * The resume is fetched and passed to the ML model for inference. * The response contains a list of job IDs and their respective match scores. * Model inference is completed and returned. Benefit/Value: * Makes the recommendation process feel intelligent and dynamic, helping users feel seen and understood by the system.	Sarah Wierzbicki	Done			3 Sprint 2
NY4-56	Create POST Process Resume	User Story: * As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can start browsing job options personalized to my background. Acceptance Criteria: * A POST endpoint /process-resume accepts file uploads via multipart/form-data. * Resume content is extracted and passed to the ML model. * The endpoint returns a JSON response containing a resumeID and a list of matched jobs. * Validation is included for file format and size. * Errors return appropriate status codes with user-friendly messages. Benefit/Value: * Allows users to initiate their personalized job search instantly through a simple upload, reducing effort and manual input.	Sarah Wierzbicki	Done			3 Sprint 2
NY4-55	Call Backend from Frontend UI	User Story: * As a job seeker, I want my resume to be analyzed as soon as I upload it, so that I can begin swiping through job matches without delay. Acceptance Criteria: * The frontend calls the Flask backend /process-resume endpoint when a resume file is selected. * On success, a resumeID is saved to local storage for session tracking. * If successful, the user can navigate to the /search page to begin swiping. Benefit/Value: * Seamless integration ensures users can quickly move from upload to results, making the app feel fast and intuitive.	Eric Lantz	Done			2 Sprint 2
NY4-54	Backend Query Job Dataset	User Story:	Sarah Wierzbicki	Done			2 Sprint 2

Sprint 1 Summary (Tasks)

Jira

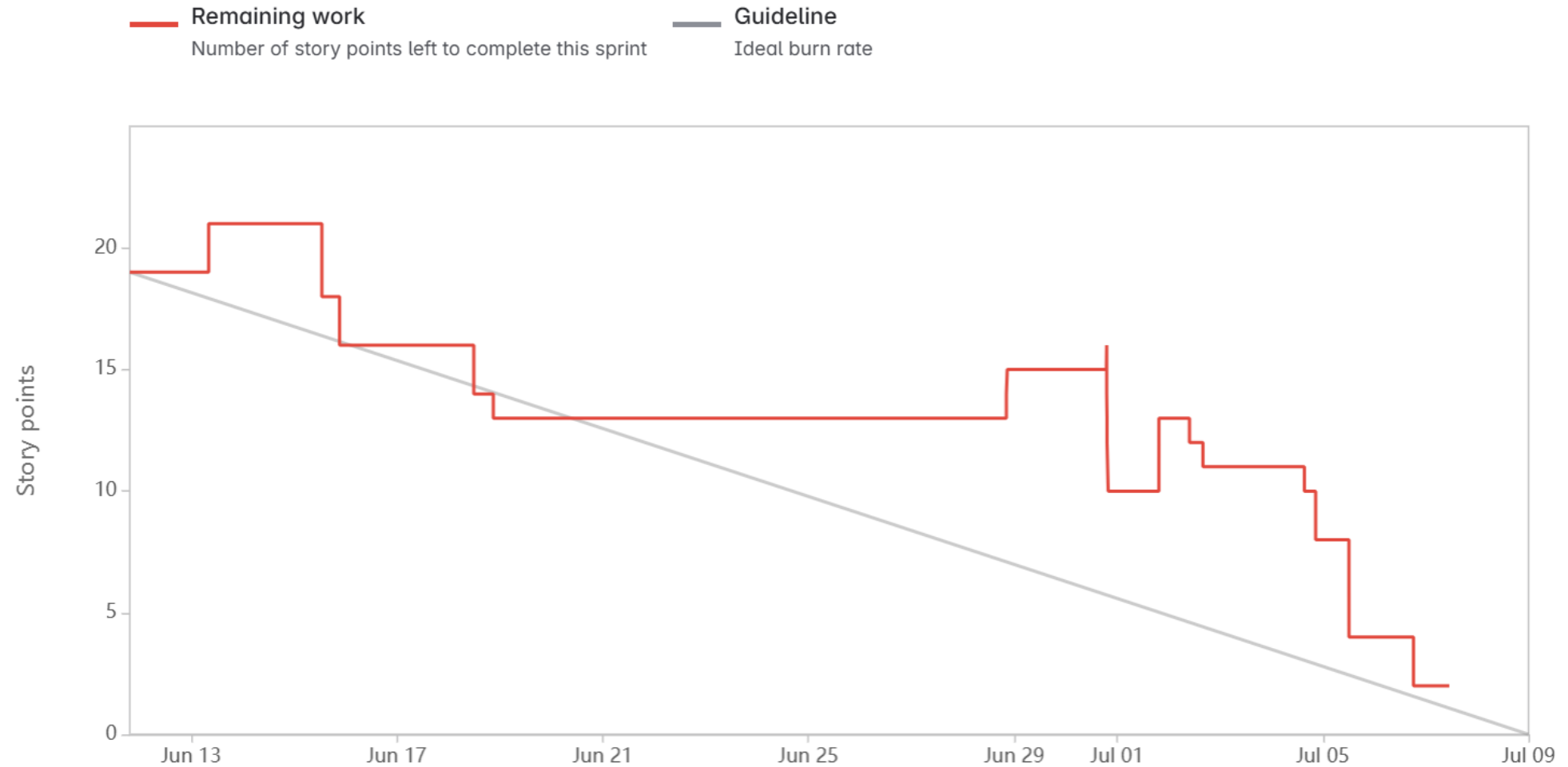
JQL Query: issueKey in (NY4-30,NY4-10,NY4-13,NY4-17,NY4-28,NY4-50,NY4-52,NY4-31,NY4-34,NY4-11,NY4-15,NY4-46,NY4-43,NY4-18,NY4-14,NY4-12,NY4-16)

Sorted by: Key descending

1-17 of 17 as at: 07/Jul/25 2:11 PM

Summary	P	Status	Resolution	Created	Due	Key	Story point estimate	Assignee
Install Docker on EC2 Instance	==	DONE	Done	28/Jun/25	09/Jul/25	NY4-52	1	Joseph Indriolo
Provision MongoDB Table on EC2 Instance	==	DONE	Done	28/Jun/25	09/Jul/25	NY4-50	1	Joseph Indriolo
Design landing page	==	DONE	Done	23/Jun/25	09/Jul/25	NY4-46	1	Bushra, Ms. Anika
Develop ResuMind's brand identity	==	DONE	Done	23/Jun/25	09/Jul/25	NY4-43	1	Bushra, Ms. Anika
Create swipe left/right action	==	DONE	Done	19/Jun/25	09/Jul/25	NY4-34	1	Eric Lantz
Create initial card component for job posting	==	DONE	Done	19/Jun/25	09/Jul/25	NY4-31	1	Eric Lantz
Create backend API using Flask	==	TO DO	Unresolved	13/Jun/25	09/Jul/25	NY4-30	2	Joseph Indriolo
Identify tool for front end design	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-28	1	Bushra, Ms. Anika
Work on job recommendation model	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-18	2	Sarah Wierzbicki
Clean job description data to be used for our database of jobs	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-17	2	Sarah Wierzbicki
Refine and complete high level architecture diagram	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-16	2	Joseph Indriolo
Begin developing code for resume parser	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-15	2	Sarah Wierzbicki
Install needed frameworks on EC2 Instance	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-14	2	Joseph Indriolo
Provision and Deploy Amazon EC2 Instance	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-13	2	Joseph Indriolo
Identify the color scheme and design aspects of the frontend	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-12	2	Bushra, Ms. Anika
Identify and install any additional react libraries needed for the frontend	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-11	1	Eric Lantz
Create react app for frontend development	==	DONE	Done	11/Jun/25	09/Jul/25	NY4-10	3	Eric Lantz

Sprint 1 Summary (Burndown Chart)



Sprint 2 – User Stories And Acceptance Criteria

A	B	C
Issue key	Summary	Description
NY4-57	Create GET Model Inference	User Story: * As a job seeker, I want the platform to generate job recommendations based on my resume's content, so that I can see how well I match open roles. Acceptance Criteria: * A GET endpoint /model-inference accepts a resumeID as part of a body. * The resume is fetched and passed to the ML model for inference. * The response contains a list of job IDs and their respective match scores. * Model inference is completed and returned. Benefit/Value: * Makes the recommendation process feel intelligent and dynamic, helping users feel seen and understood by the system.
NY4-56	Create POST Process Resume	User Story: * As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can start browsing job options personalized to my background. Acceptance Criteria: * A POST endpoint /process-resume accepts file uploads via multipart/form-data. * Resume content is extracted and passed to the ML model. * The endpoint returns a JSON response containing a resumeID and a list of matched jobs. * Validation is included for file format and size. * Errors return appropriate status codes with user-friendly messages. Benefit/Value: * Allows users to initiate their personalized job search instantly through a simple upload, reducing effort and manual input.
NY4-55	Call Backend from Frontend UI	User Story: * As a job seeker, I want my resume to be analyzed as soon as I upload it, so that I can begin swiping through job matches without delay.

Metrics

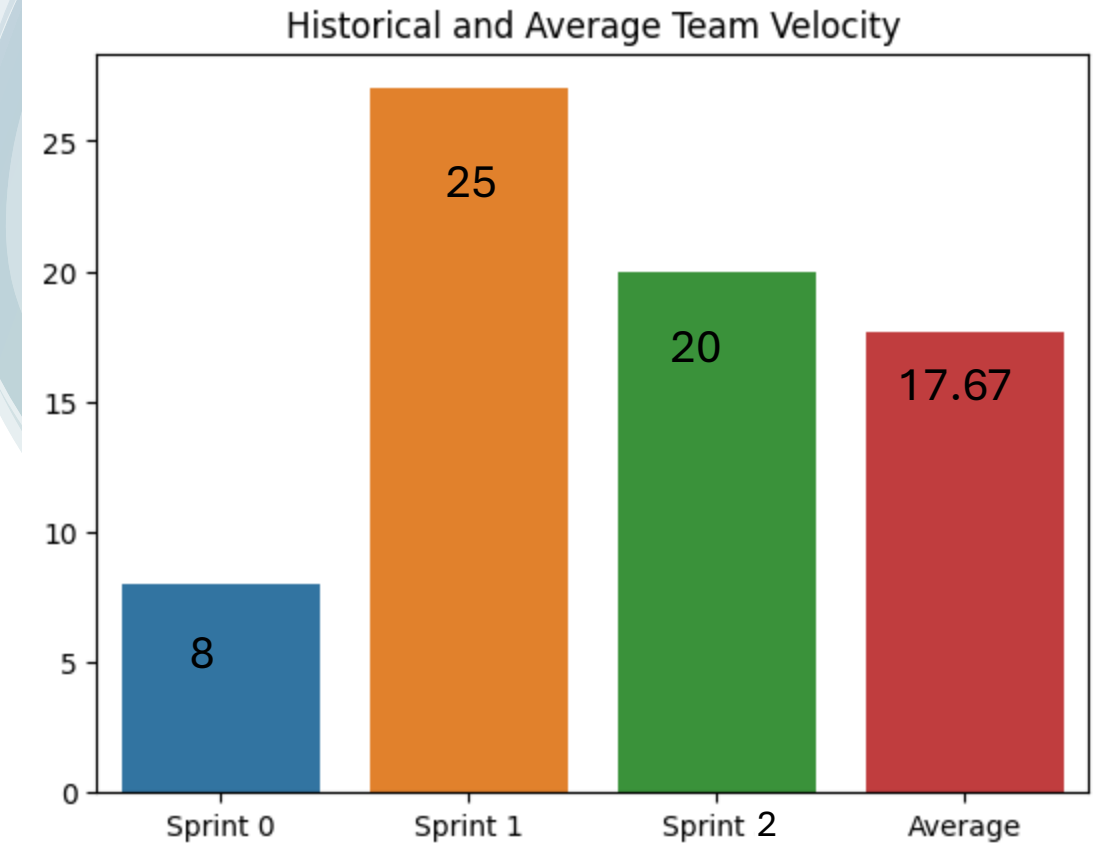


Key Items:

- Team Velocity
- Burndown chart
- Completed/Committed Ratio

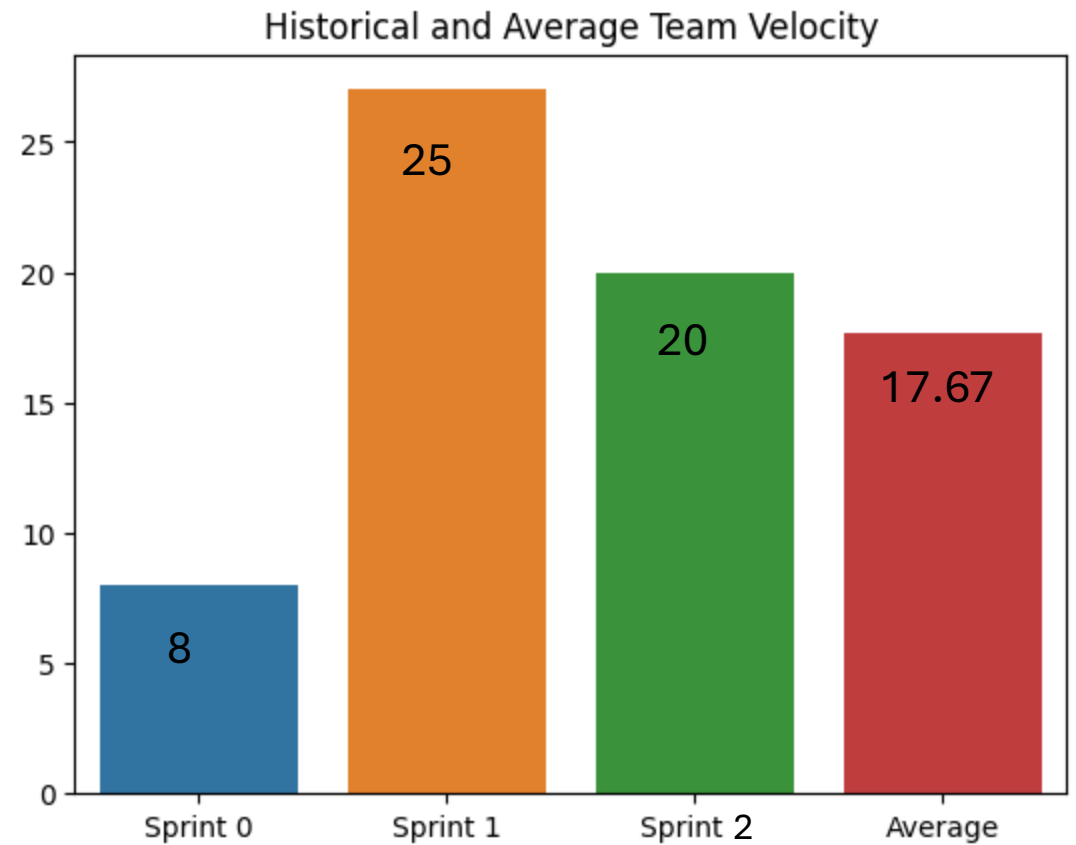
Metrics: Team Velocity for Sprint 2

- Sprint 2: Initial committed story points: **20 points**
- Sprint 2: Final committed story points: **20 points**
 - Total points carried over: 2 points



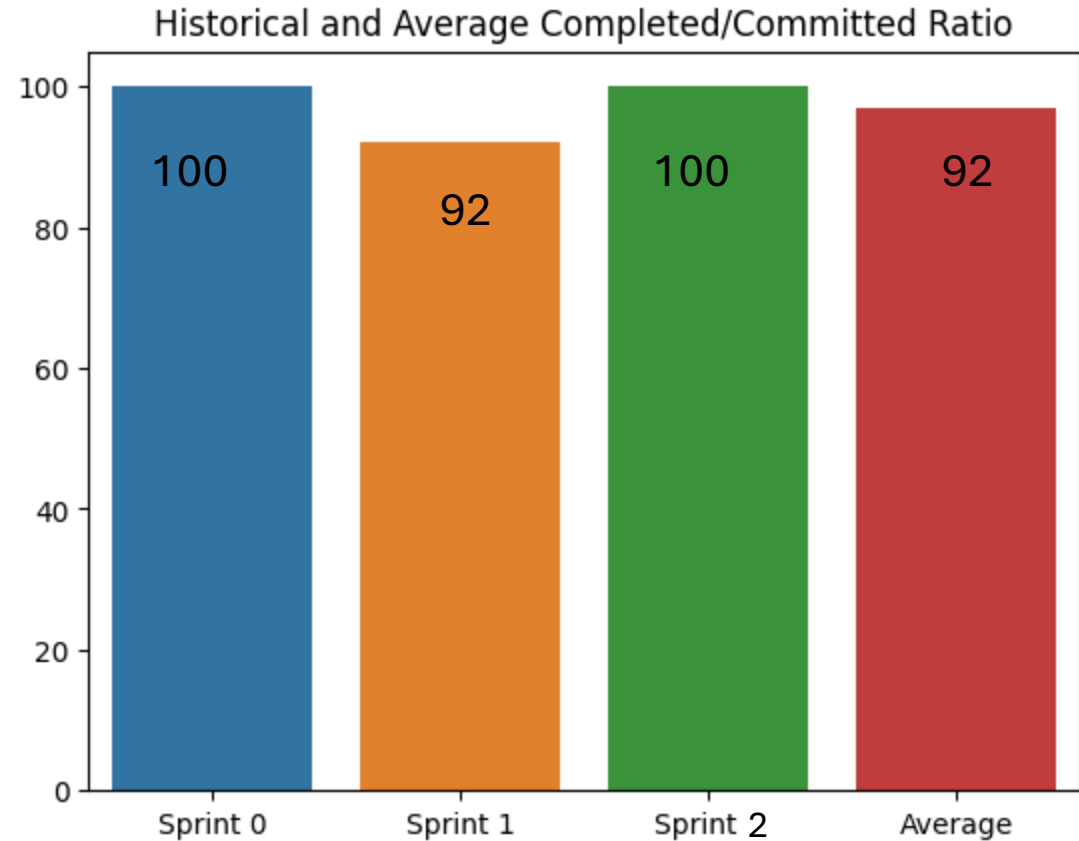
Metrics: Historical and Average Team Velocity

- Our team velocity over the course of the project averaged to 17.67 story points
- Sprint 0: 8 completed story points
- Sprint 1: 25 completed story points
- Sprint 2: 20 completed story points



Metrics: Historical and Average Completed/Committed Ratio

- Our completed/committed ratio over the course of the project averaged to 92%.
 - Sprint 0: 100%
 - Sprint 1: 92%
 - Sprint 2: 100%
- There was one incomplete task from Sprint 1 that was rolled over and completed in Sprint 2



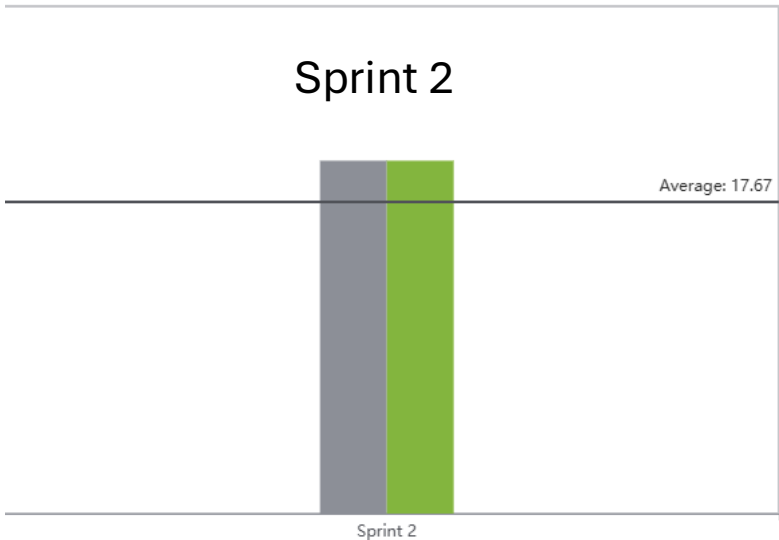
Sprint 2 Committed/Completed Ratio

Commitment

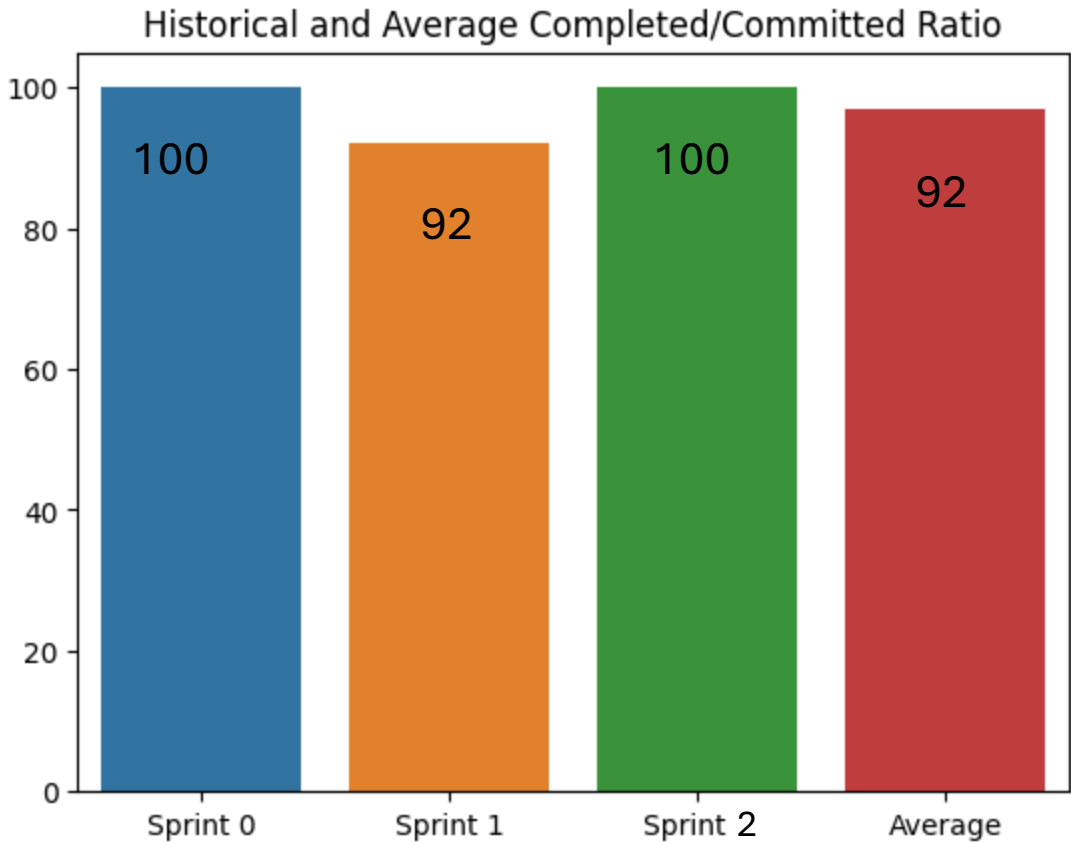
The amount of work in the sprint when it began.

Completed

The amount of work done during the sprint.



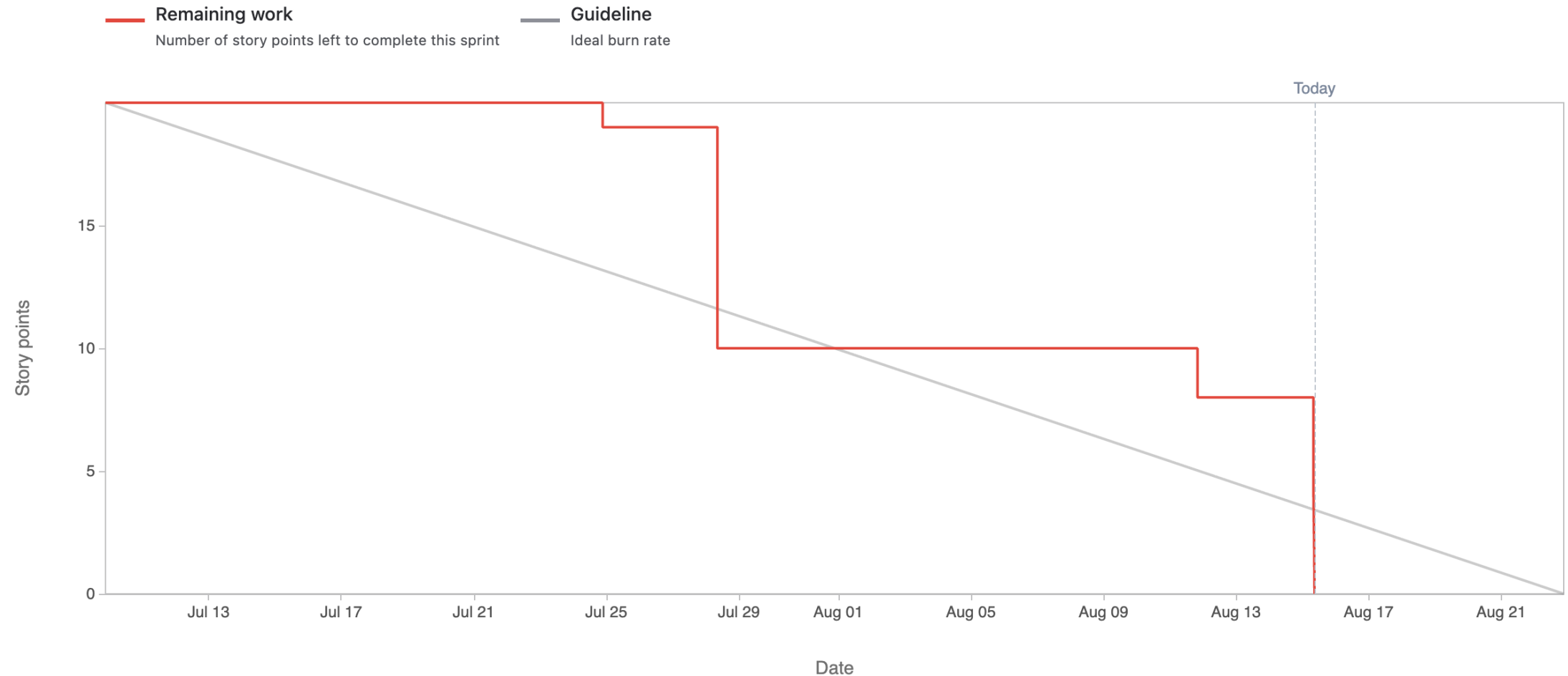
$$20/20 * 100 = 100\%$$



Metrics: Sprint 2 Burndown Chart

Date - July 9th, 2025 - August 22nd, 2025

Sprint goal - Achieve MVP (Working frontend, functional backend ML model, connected and working together via Flask API, on the Ec2 instance).



Project Demo

ResuMind

Home Job Search Matches



ResuMind

It's like Tinder... but for job searching!

Whether you are actively or passively searching for a job, looking for new opportunities can be overwhelming due to:

- High volume of listings on job sites such as Indeed, LinkedIn, Monster
- Lack of matching/relevant skills
- Fake job postings

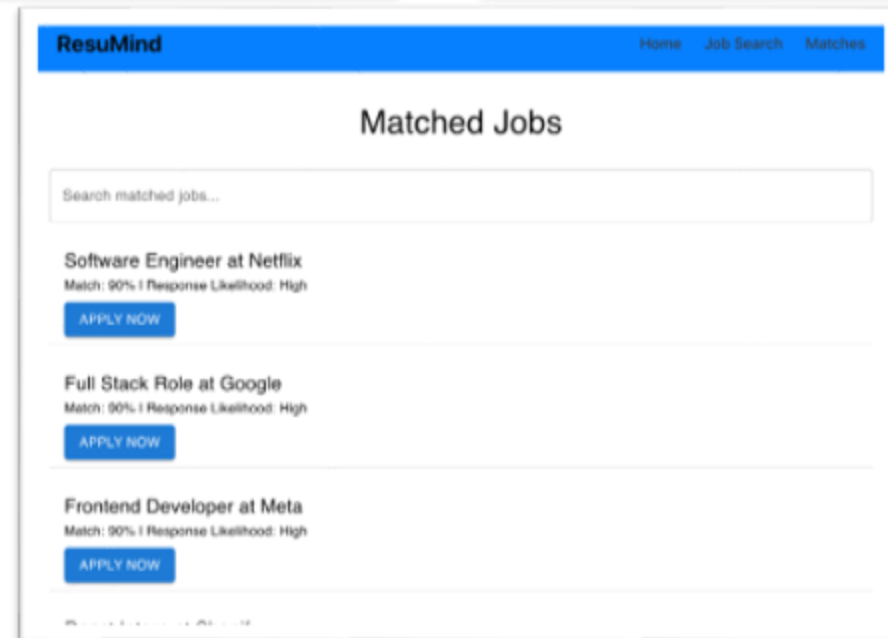
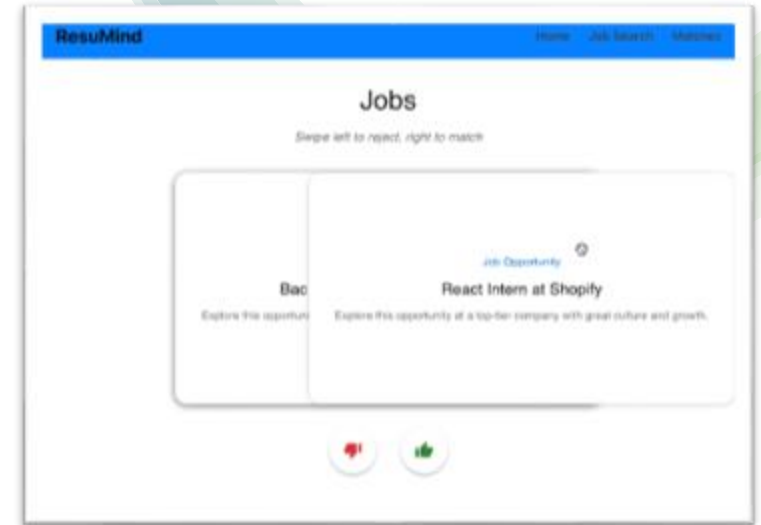
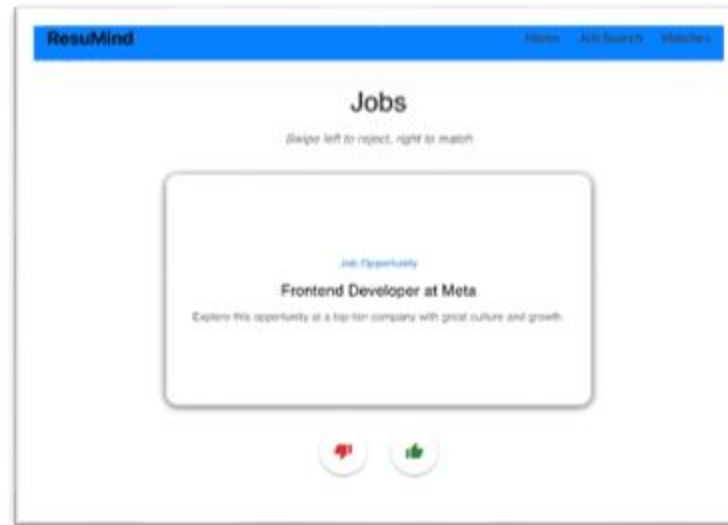
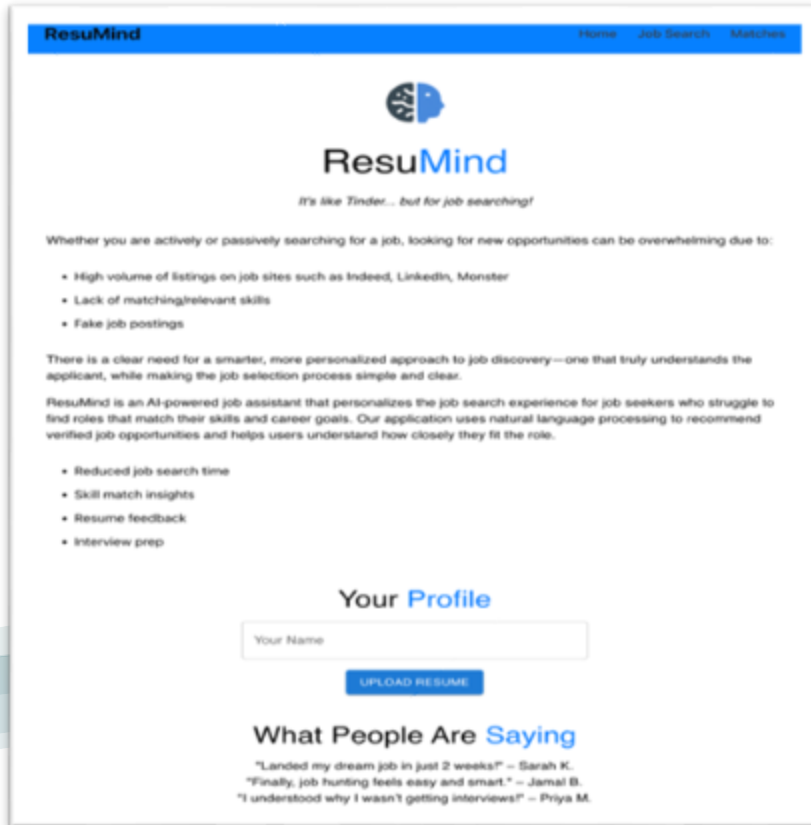
There is a clear need for a smarter, more personalized approach to job discovery—one that truly understands the applicant, while making the job selection process simple and clear.

ResuMind is an AI-powered job assistant that personalizes the job search experience for job seekers who struggle to find roles that match their skills and career goals. Our application uses natural language processing to recommend verified job opportunities and helps users understand how closely they fit the role.

- Reduced job search time
- Skill match insights



Website Screenshots



Test Cases

Tr	Feature Area	Component	Tr	User Story	Scenario	Acceptance Criteria	Trigger/Interaction	Expected Behavior	Status
Backend ML	SageMaker Endpoint	...	As a job seeker, I want accurate and fast job recommendations based on my resume, so that I can quickly discover relevant job opportunities.	Endpoint accepts resume input and returns prediction	The trained ML model is deployed as a SageMaker endpoint that accepts resume input and returns job recommendations.	Send a valid resume payload to the SageMaker HTTPS endpoint	HTTP 200 with JSON including predicted job category and a list of job recommendations	Pass	
Backend ML	SageMaker Endpoint	...	...	Endpoint requires secure access	The endpoint is accessible via a secure HTTP API.	Invoke endpoint over HTTPS with and without valid auth/credentials	HTTPS succeeds with valid credentials; unauthorized requests receive 401/403	Pass	
Backend ML	SageMaker Endpoint	...	...	Typical prediction completes under 1 second	The model returns predictions within 1 second for a typical resume file.	Post a typical-size resume (≤200KB PDF/text) and measure latency	p95 latency ≤ 1000 ms for response	Pass	
Backend ML	SageMaker Endpoint	...	...	Output contains IDs, scores, and metadata	Model output includes a list of job IDs, match percentages, and any additional metadata used by the frontend.	Send a valid resume and inspect response schema	Response JSON includes non-empty list of objects with jobId, matchPct, and metadata fields	Pass	
Backend ML	SageMaker Endpoint	...	...	Monitoring and graceful error handling in place	Model uptime is monitored, and errors are logged or handled gracefully.	Induce a handled error (e.g., malformed payload) and check logs/metrics	Request returns a clear error (4xx/5xx as appropriate); error is logged; endpoint health remains OK	Pass	
Backend API	Flask Endpoint	...	As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can get personalized recommendations without needing technical knowledge.	/process-resume accepts file upload	Flask API exposes an endpoint /process-resume that accepts file uploads via POST.	POST multipart/form-data with 'file' to /process-resume	HTTP 200; response body includes resumeID and list of matched jobs	Pass	
Backend API	Flask Endpoint	...	...	Resume forwarded to ML model and results returned	The uploaded resume is forwarded to the ML model and the results are returned in JSON format.	Mock/stub model to return known IDs; call /process-resume	Response JSON contains the same job IDs and match data from the model	Pass	
Backend API	Flask Endpoint	...	...	Response contains resumeID and matched jobs	API response includes a resumeID and list of matched jobs.	Call /process-resume successfully	Response includes non-empty resumeID string and matches array	Pass	
Backend API	Flask Endpoint	...	...	CORS enabled for frontend domain	CORS is enabled so the frontend can make cross-origin requests.	Browser preflight and POST from frontend origin	CORS headers present; preflight and POST succeed without CORS errors	Pass	
Backend API	Flask Endpoint	...	...	Correct status codes on success and errors	Returns appropriate status codes for success (200), client errors (400), or server errors (500).	Submit valid request; missing file; trigger server error	Valid returns 200; missing/invalid returns 400; server fault returns 500 with safe error body	Pass	
Frontend	Browser Compatibility	...	As a user, I should be able to visit and use the website from any browser without errors.	Load and basic navigation across browsers	Ensure ResuMind works across browsers.	Open app in latest Chrome, Safari and navigate Home → Search → Matches	Each page loads and is usable with no blocking console errors	Pass	
Frontend	Browser Compatibility	...	...	File upload works in common browsers	Ensure file upload works across browsers.	Upload PDF/DOC/DOCX in each browser	Valid files are accepted and processed consistently; invalid types show validation message	Pass	
Frontend	Browser Compatibility	...	...	Responsive layout remains intact	Website looks correct across screen sizes.	Resize viewport to 1440, 1024, 390 widths	No overlapping/clipped text; navbar/cards remain accessible	Pass	
Frontend	Browser Compatibility	...	...	Swipe and click interactions behave consistently	Swiping/clicking works in all browsers.	Swipe cards and click actions on each browser	Interactions execute with the same behavior and visual feedback	Pass	
Frontend	Form Inputs	...	As a new user, I want to enter my name and upload my resume easily, so that I can quickly get started and receive job suggestions personalized to me.	Homepage shows name field and resume upload	The homepage includes a text input for name and a resume upload button.	Load homepage	Name input and upload control are visible and focusable	Pass	
Frontend	Form Inputs	...	...	Accept only supported file types	The resume input accepts .pdf, .doc, and .docx files.	Try uploading .pdf/.doc/.docx then .txt/.png	Supported types accepted; unsupported types blocked with a friendly message	Pass	
Frontend	Form Inputs	...	...	Selected file stored and ready to send	Upon selection, the file is stored and ready to be sent to the backend.	Select a valid file and inspect client state	File object is present in state and attached for submission	Pass	
Frontend	Form Inputs	...	...	Optional name personalizes UI text	The name input can be optionally used to personalize UI text.	Enter a name and continue; repeat with empty name	Greeting/personalized text appears when provided; app still works without a name	Pass	
Frontend	Backend Call	...	As a job seeker, I want my resume to be analyzed as soon as I upload it, so that I can begin swiping through job matches without delay.	Auto POST to /process-resume on file select	The frontend calls the Flask backend /process-resume endpoint when a resume file is selected.	Choose a resume file on the upload control	Frontend sends POST /process-resume and shows loading state	Pass	
Frontend	Backend Call	...	...	Save resumeID for session tracking	On success, a resumeID is saved to local storage for session tracking.	Complete a successful upload and inspect storage	resumeID is saved in localStorage/sessionStorage	Pass	
Frontend	Backend Call	...	...	Navigate to search after success	If successful, the user can navigate to the /search page to begin swiping.	After receiving a 200 response	App routes to /search and job cards are visible	Pass	
Frontend	Job Card Component	...	As a job seeker, I want each job card to clearly display relevant job details (title, description, match score, etc.), so that I can make informed decisions while swiping.	Render with all expected props	The job card component accepts props for title, description, matchScore, and likelihood.	Render a card with those four props	Each field displays correctly; no overflow/clipping	Pass	
Frontend	Job Card Component	...	...	Props populated from backend data	Props are dynamically populated from job data received from the backend.	Load a jobs response and map to card props	Cards show correct values corresponding to data	Pass	
Frontend	Job Card Component	...	...	Graceful fallbacks when data missing	Cards render correctly with various prop values and show placeholder text if a prop is missing.	Omit one prop at a time and render	Placeholder text appears; card does not crash	Pass	
Frontend	Job Card Component	...	...	Reusable across pages	The component is reusable across the swipe and matches pages.	Render the same component on Swipe and Matches views	Visuals/behavior consistent across both pages	Pass	
Frontend	Job Card Component	...	...	Test card renders during development	A test card renders correctly with mock data during development.	Render with mock job data	Card renders as designed with mock content	Pass	

Test Cases

Tr	Feature Area	Component	Tr	User Story	Scenario	Acceptance Criteria	Trigger/Interaction	Expected Behavior	Status
					development	during development			
	Backend	Job Dataset Query		As a job seeker, I want the platform to match me with real job postings based on my resume, so that the recommendations I receive are relevant and up-to-date.	Backend connects to configured dataset	Backend connects to a structured dataset (CSV, database, or API) of job postings.	Startup/connect and perform a sample query	Query returns ≥1 job from configured source	Pass
	Backend	Job Dataset Query		...	Retrieve relevant jobs by skills/keywords/embedding	A function is implemented to retrieve relevant jobs based on skills, keywords, or embedding similarity.	Request recommendations with known skills in resume	Top results include those skills or close semantic matches	Pass
	Backend	Job Dataset Query		...	Only verified or curated jobs are returned	Only verified or curated jobs are returned.	Include an unverified item in test data and query	Unverified item is not present in results	Pass
	Backend	Job Dataset Query		...	Return at least 20 jobs per request	At least 20 jobs are returned per resume request.	Perform a standard resume request	Response includes 20 or more job objects (or first page size ≥20)	Pass
	Backend	Job Dataset Query		...	Each returned job has required fields	Returned job objects include id, title, description, and metadata used by the frontend.	Inspect one returned object	Object includes id, title, description, and metadata fields	Pass
	Project Setup	Test Case Sheet		As a developer, I want a centralized test case sheet that outlines how each feature will be tested, so that I can systematically verify functionality and ensure the app works as expected.	Current stories have tests	Test cases are written for the currently implemented user stories.	Review the sheet rows	Each sprint story has at least one corresponding test case	Pass
	Project Setup	Test Case Sheet		...	Sheet stored in shared location	The sheet is stored in a shared location (shared drive).	Open sheet from shared drive path	Path is shared; teammates can access it	Pass
	Project Setup	Test Case Sheet		...	Team can contribute tests	The team has access and can contribute additional test cases as features are developed.	Teammate attempts to add/edit rows	Changes save successfully and appear in version history	Pass
	Backend API	Process Resume		As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can start browsing job options personalized to my background.	Accept multipart file uploads	A POST endpoint /process-resume accepts file uploads via multipart/form-data.	POST multipart/form-data with 'file' field	HTTP 200 with JSON {resumeId, matches:[...]}	Pass
	Backend API	Process Resume		...	Extract resume content and pass to model	Resume content is extracted and passed to the ML model.	Upload a PDF containing a known keyword/skill	Extraction contains the keyword; payload sent onward to model	Pass
	Backend API	Process Resume		...	Return resumeId and matched jobs	The endpoint returns a JSON response containing a resumeId and a list of matched jobs.	Successful request to /process-resume	Response includes non-empty resumeId and matches array	Pass
	Backend API	Process Resume		...	Validate format and size	Validation is included for file format and size.	Upload unsupported or oversized file	HTTP 400/413 with user-friendly validation message	Pass
	Backend API	Process Resume		...	Graceful error responses	Errors return appropriate status codes with user-friendly messages.	Force extraction failure or model error	HTTP 500 with safe message; no stack traces leaked	Pass
	Backend API	Model Inference		As a job seeker, I want the platform to generate job recommendations based on my resume's content, so that I can see how well I match open roles.	Accept resumeId and return predictions	A GET endpoint /model-inference accepts a resumeId as part of a body.	Call /model-inference with a resumeId (query/body to match implementation)	HTTP 200 with list of job IDs and their match scores	Pass
	Backend API	Model Inference		...	Resume is fetched	The resume is fetched and passed to the ML model for inference.	Call the /model-inference endpoint with a valid resumeId that was previously generated by /process-resume	Model runs and returns job IDs with match scores	Pass
	Backend API	Model Inference		...	Response includes job IDs and their match scores	The response contains a list of job IDs and their respective match scores.	Call /model-inference with a valid resumeId	JSON returns list of job IDs with match scores	Pass
	Backend API	Model Inference		...	Run model inference	Model inference is completed and returned.	Invoke inference for a valid stored resumeId	Model runs successfully and returns scores	Pass
	Project Setup	Deployment Research		As a developer working on ResuMind, I want to identify the best deployment strategy for the backend and ML model, so that users experience fast and reliable service.	Compare at least two methods	At least 2 deployment methods are evaluated (e.g., AWS SageMaker, Lambda, EC2)	Open the research report	Report shows evaluation of 2+ methods including cost and Flask integration	Pass
	Project Setup	Deployment Research		...	Clear recommendation recorded	Recommendations are documented clearly in a markdown or PDF report.	Go to the conclusion of the report	One method is recommended with rationale and risks	Pass
	Project Setup	Deployment Research		...	Team agreement for MVP	Team agrees on a method based on findings for MVP launch.	Check team notes/meeting outcomes	Decision captured and linked for reference	Pass

Sprint 2 Completed Stories

☒ NY4-49 Deploy ML model to Sagemaker	BACKEND - ML MODEL	DONE ▾	2	JL
☒ NY4-30 Create backend API using Flask	BACKEND - ML MODEL	DONE ▾	1	SW
☒ NY4-48 Cross browser testing	FRONTEND - UI DEVE...	DONE ▾	1	BA
☒ NY4-47 Implement Form Inputs	FRONTEND - UI DEVE...	DONE ▾	1	BA
☒ NY4-55 Call Backend from Frontend UI	FRONTEND - UI DEVE...	DONE ▾	2	EL
☒ NY4-33 Create props for each part of the job application card	FRONTEND - UI DEVE...	DONE ▾	2	EL
☒ NY4-54 Backend Query Job Dataset	BACKEND - ML MODEL	DONE ▾	2	SW
☒ NY4-19 Create test cases sheet	TEAM PROCESSES & ...	DONE ▾	2	BA
☒ NY4-56 Create POST Process Resume	BACKEND - ML MODEL	DONE ▾	3	SW
☒ NY4-57 Create GET Model Inference	BACKEND - ML MODEL	DONE ▾	3	SW
☒ NY4-29 Research Deployment Methods	BACKEND - ML MODEL	DONE ▾	1	JL

Sprint 2 Retrospective

The screenshot shows a Jira Sprint Retrospective Board for 'NY4 Sprint 2 Retrospective'. The board is divided into four columns: 'Went Well', 'Could Be Improved', 'Action Items', and 'Questions & Discussion'. Each column contains several cards with text describing the team's performance and plans for improvement. The background of the board features a mountain landscape.

NY4 Sprint 2 Retrospective

- Went Well**
 - Reached MVP!!!
 - Separated Work very well, in an efficient way. KT to each other.
 - All the tasks we assigned did not exceed capacity.
 - When problems arose, we worked through them quickly and efficiently.
 - Addressed action items from last sprint
 - Communicated more clearly, got
- Could Be Improved**
 - Give more updates on each persons parts.
 - Be more sharing of knowledge you may be an SME on.
 - Better discovery of cloud infa needs.
 - More clear rubrics for papers, etc.
- Action Items**
 - Give more updates, clear, often.
 - Less tunnel vision on each respesctive part.
 - Better preliminary research concerning cloud needs.
 - Be more proactive about clarifying rubrics, and class requirements.
- Questions & Discussion**
 - + Add a card

[Sprint Retrospective Board](#)



Live Application Demo

Wiki Link:

<https://github.com/htmhw/2025Su-NY4/wiki>